Assignment-01

Name:	Ronit Rout	**Serial Number:**	38
Registration Number:	24E118B08	**Assignment Number:**	01
Section:	24E1B1	**Date:**	30-09-2026

Question 1

```
import numpy as np
data = np.array([[10, -20, 30], [40, -50, 60], [70, 80, -90]])
row_means = np.mean(data, axis=1)
col_means = np.mean(data, axis=0)
row_std = np.std(data, axis=1)
max_col = np.max(data, axis=0)
min_row = np.min(data, axis=1)
print("Row means:", row_means)
print("Column means:", col_means)
print("Row standard deviations:", row_std)
print("Maximum values in each column:", max_col)
print("Minimum values in each row:", min_row)
```

Output

```
Row means: [ 6.66666667 16.66666667 20.        ]
Column means: [40.          3.33333333  0.        ]
Row standard deviations: [20.54804668 47.84233365 77.88880964]
Maximum values in each column: [70 80 60]
Minimum values in each row: [-20 -50 -90]
```

Question 2:

```
A = np.array([ [2, -4], [-6, 8]])
B = np.array([ [1, -3], [-5, -7]])

print("A+B:")
print(A + B)
print("A-B:")
print(A - B)
print("A.B:")
print(A @ B)
print("Transpose of A:")
print(A.T)
print("Determinant of A:")
print(np.linalg.det(A))
print("Inverse of A:")
print(np.linalg.inv(A))
```

output:

```
A+B:
[[  3  -7]
 [-11   1]]

A-B:
[[ 1 -1]
 [-1 15]]

A.B:
[[ 22  22]
 [-46 -38]]

Transpose of A:
[[ 2 -6]
 [-4  8]]

Determinant of A:
-8.000000000000002

Inverse of A:
[[-1.   -0.5 ]
 [-0.75 -0.25]]
```

Question 3:

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import random
import pandas as pd
from scipy import stats

import pandas as pd
data={

'product':['laptop','Smartphone','Monitor','Headphone','Mo
use'],
    'Price':[200000,180000,15000,32000,5000],
    'Sales':[150000,120000,10000,12000,1000],
    'profit':[50000,20000,5000,5000,2000]
}
df=pd.DataFrame(data)
print("Dataframe:\n",df)
print("\nColumn Name:",df.columns)
print("\nNumber Of Rows & columns :",df.shape)
print("\nFirst 3 Rows:\n",df.head(3))
print("\nOnly Sales And Profit
Column:\n",df[['Sales','profit']])
print("\nAverage Sales:", df['Sales'].mean())
print("Maximum Sales:", df['Sales'].max())
print("Minimum Sales:", df['Sales'].min())
print("\nProducts with Profit > 4000:\n",df[df['profit'] >
4000]['product'])
df.sort_values('profit',ascending=False)
print("\nDataframe sorted by Profit (Descending):\n",df)
```

output:

```
Dataframe:
       product   Price   Sales  profit
0      laptop  200000  150000   50000
1  Smartphone  180000  120000   20000
2     Monitor   15000   10000    5000
3   Headphone   32000   12000    5000
4       Mouse    5000    1000    2000

Column Name: Index(['product', 'Price', 'Sales', 'profit'], dtype='str')

Number Of Rows & columns : (5, 4)

First 3 Rows:
       product   Price   Sales  profit
0      laptop  200000  150000   50000
1  Smartphone  180000  120000   20000
2     Monitor   15000   10000    5000

Only Sales And Profit Column:
     Sales  profit
0  150000   50000
1  120000   20000
2   10000    5000
3   12000    5000
4    1000    2000

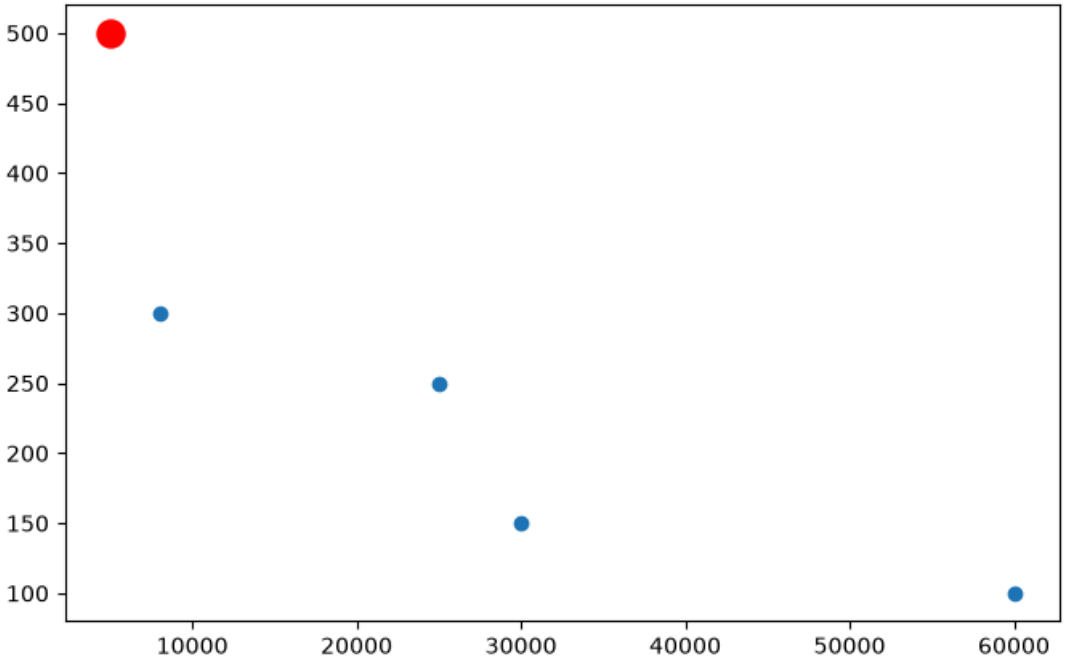
Average Sales: 58600.0
Maximum Sales: 150000
Minimum Sales: 1000

Products with Profit > 4000:
 0        laptop
1    Smartphone
2       Monitor
3     Headphone
Name: product, dtype: str
Dataframe sorted by Profit (Descending):
       product   Price   Sales  profit
0      laptop  200000  150000   50000
1  Smartphone  180000  120000   20000
2     Monitor   15000   10000    5000
3   Headphone   32000   12000    5000
4       Mouse    5000    1000    2000
```

Question 4:

```
plt.figure(figsize=(8, 5))
plt.scatter(df['Price'], df['Sales'], marker='o',
label='Products')
High_ind = df['Sales'].idxmax()
plt.scatter(
    df.loc[High_ind,'Price'],
    df.loc[High_ind,'Sales'],
    marker='o',
    color='red',
    s=150,
    label='Highest Sales'
)
```

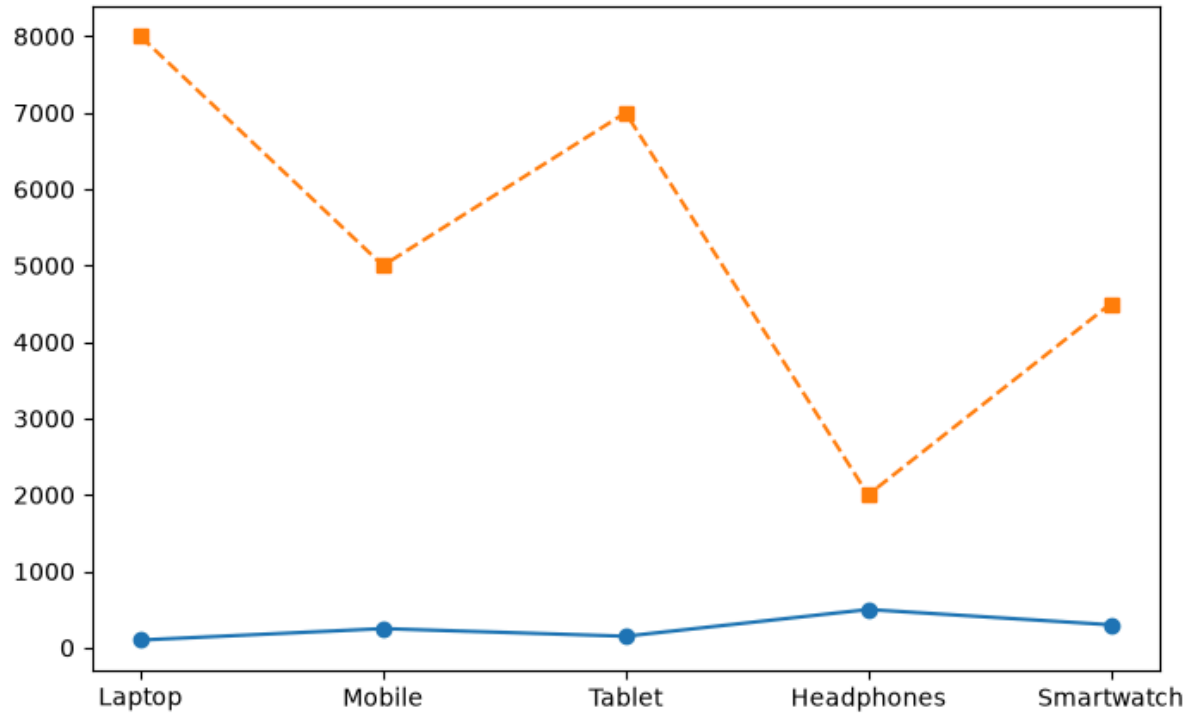
output:



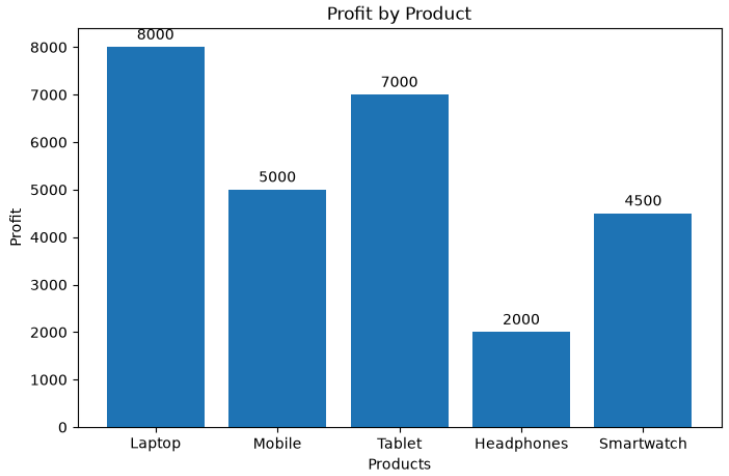

```
plt.figure(figsize=(8, 5))
plt.plot(
    df['Products'],
    df['Sales'],
    marker='o',
    linestyle='-',
    label='Sales'
)
plt.plot(
```

```
    df['Products'],
    df['Profit'],
    marker='s',
    linestyle='--',
    label='Profit'
)
```

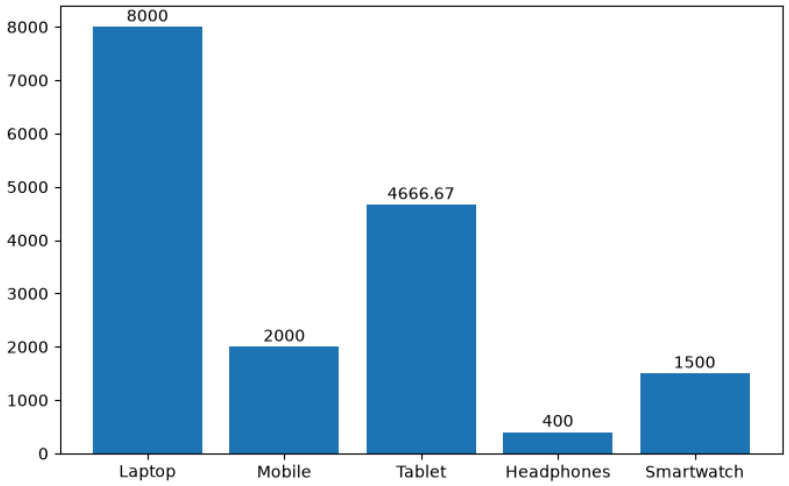
output:



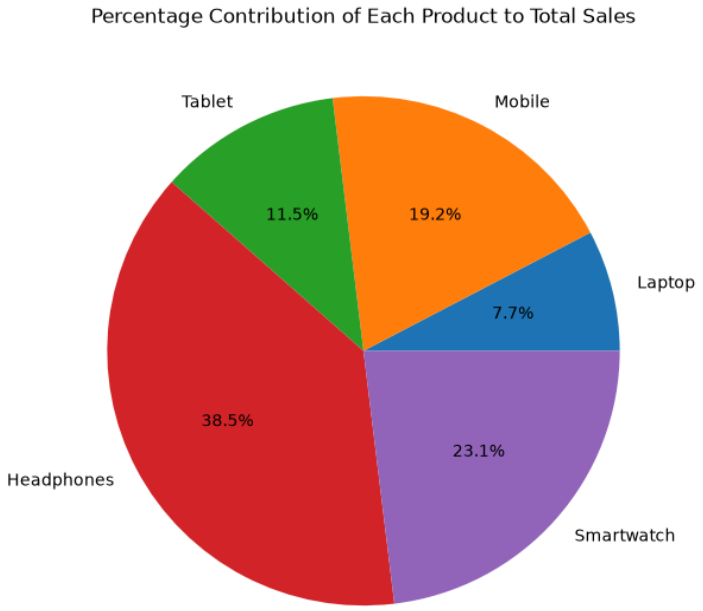
```
plt.figure(figsize=(8, 5))
bars = plt.bar(df['Products'], df['Profit'])
plt.bar_label(bars, padding=3)
plt.xlabel('Products')
plt.ylabel('Profit')
plt.title('Profit by Product')
plt.show()
```



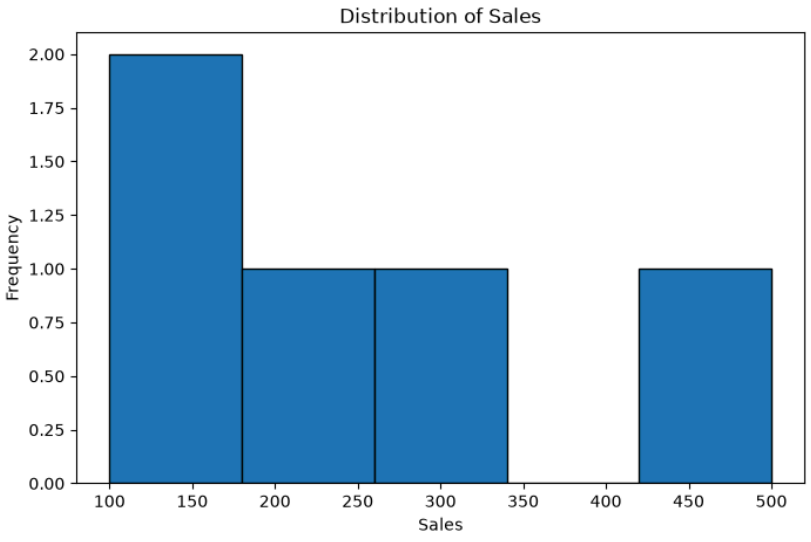

```
profit_margin = df['Profit'] / df['Sales'] * 100
plt.figure(figsize=(8, 5))
bars = plt.bar(df['Products'], profit_margin)
plt.bar_label(bars, padding=1)
```



```
plt.figure(figsize=(7, 7))
plt.pie(
    df['Sales'],
    labels=df['Products'],
    autopct='%1.1f%%'
)
plt.title('Percentage Contribution of Each Product to Total Sales')
```




```
plt.figure(figsize=(8, 5))
plt.hist(df['Sales'], bins=5, edgecolor='black')
plt.xlabel('Sales')
plt.ylabel('Frequency')
plt.title('Distribution of Sales')
```



Question 5:

Question 5:

```
ns = 100  # number of samples
sd = np.random.randint(1, 101, ns)
ts = 4 * sd + np.random.normal(0, 20, ns)
df = pd.DataFrame({
    "StudyHour": sd,
    "TestScore": ts
})
print("Complete Dataset:")
print(df)
```

output:

```
Complete Dataset:
    StudyHour   TestScore
0          23   70.017001
1          97  369.092456
2          33  154.235526
3          67  283.519318
4          48  174.013522
..        ...         ...
95         40  165.710599
96         79  313.371301
97         53  211.760164
98        100  416.513311
99         78  281.066343

[100 rows x 2 columns]
```

```
print("\nNumber of samples:", df.shape[0])
print("Number of features:", 1)
print("Number of target variables:", 1)
print("Feature name:", "StudyHour")
print("Target name:", "TestScore")
print("Shape of dataset:", df.shape)
print("Data type of feature:", df["StudyHour"].dtype)
```

Output:

```
Number of samples: 100
Number of features: 1
Number of target variables: 1
Feature name: StudyHour
Target name: TestScore
Shape of dataset: (100, 2)
Data type of feature: int32
```

```
print("\nStatistics:")
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())
```

output:

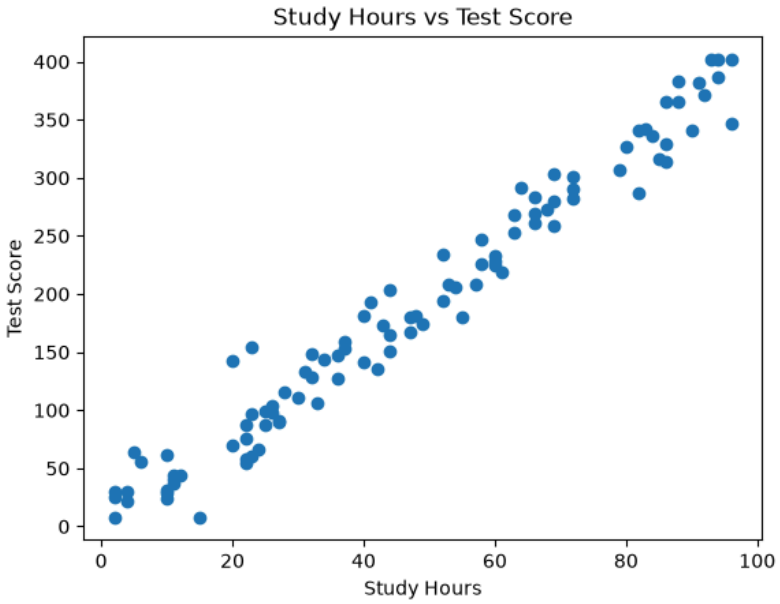
```
Statistics:
        StudyHour   TestScore
count  100.000000  100.000000
mean    53.580000  213.995921
std     29.641351  119.179460
min      1.000000  -27.379690
25%     31.500000  132.903044
50%     56.000000  215.530109
75%     78.000000  310.176793
max    100.000000  416.513311

Missing Values:
StudyHour    0
TestScore    0
dtype: int64
```

```
plt.scatter(df["StudyHour"], df["TestScore"])

plt.title("Study Hours vs Test Score")
plt.xlabel("Study Hours")
plt.ylabel("Test Score")
```

output:



Question 6:

```
import numpy as np
import matplotlib.pyplot as plt
import random
import pandas as pd
from scipy import stats

ns = 100  # number of samples
sd = np.random.randint(1, 101, ns)
ts = 4 * sd + np.random.normal(0, 1, ns)
df = pd.DataFrame({
    "StudyHour": sd,
    "TestScore": ts
})
print("Complete Dataset:")
print(df)
```

Output:

```
Complete Dataset:
    StudyHour   TestScore
0          71  283.763757
1          60  239.234962
2          17   68.314520
3          76  301.850498
4          61  243.552279
..        ...         ...
95         98  391.593000
96         71  284.978234
97         21   85.126648
98         55  219.032705
99          5   20.372455

[100 rows x 2 columns]
```

```
x=df.StudyHour
y=df.TestScore

from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(x,y,test_size=20,random_stat
e=42)
print("Number of training samples:", len(xtrain))
print("Number of testing samples:", len(xtest))
```

Output:

```
Number of training samples: 80
Number of testing samples: 20
```

```
#model creationlinear regression predictions.csv
from sklearn import linear_model
lr = linear_model.LinearRegression()
xtrain=xtrain.values.reshape(80,1)
xtest=xtest.values.reshape(20,1)
#training
```

```
lr.fit(xtrain, ytrain)
cf = lr.coef_[0].round(2)
intc = lr.intercept_.round(2)
print("\nSlope (Coefficient):", cf)
print("Intercept:", intc)
#Equation of the line
equ=f"TestScore={cf}*StudyHour+{intc}"
print("\nRegression Equation:", equ)
```

Output:

```
Slope (Coefficient): 4.0
Intercept: 0.06

Regression Equation: TestScore=4.0*StudyHour+0.06
```

```
#model testing
ypred=lr.predict(xtest)
print(ypred)
print(ytest)
```

output:

```
[164.00415377 148.0097177    4.05979311 267.9679882  140.01249967
 303.95546935 271.96659721  96.02780049 375.93043165 327.94712345
 391.92486771   8.05840212 303.95546935  32.05005622 331.94573246
 164.00415377 271.96659721 271.96659721  96.02780049 259.97077017]
83    164.331676
53    147.836159
70      3.856084
45    268.297025
44    141.233214
39    304.347196
22    272.974342
80     96.007742
10    376.601361
0     328.652758
18    390.559183
30      8.219265
73    302.352951
33     32.488007
90    331.715916
4     163.590860
76    272.565676
77    272.308146
12     96.014687
31    259.263208
Name: TestScore, dtype: float64
```

```
from sklearn import metrics
RSS=np.sum((ytest-ypred)**2).round(2)
print("RSS:",RSS)
MAE=metrics.mean_absolute_error(ytest,ypred)
print("MAE:",MAE)
MSE=metrics.mean_squared_error(ytest,ypred)
print("MSE:",MSE)
RMSE=np.sqrt(MSE)
print("RMSE:",RMSE)
TSS=np.sum((ytest-np.mean(ytest))**2).round(2)
print("TSS:",TSS)
R2=(1-(RSS/TSS)).round(2)
print("R2 value:",R2)
```

output:

```
RSS: 9.75
MAE: 0.5461032161852095
MSE: 0.48727987416688334
RMSE: 0.6980543490064963
TSS: 277754.91
R2 value: 1.0
```

```
df2 = pd.DataFrame({
    "StudyHour": xtest[:, 0],
    "Y-Actual": ytest.round(2),
    "Y-Predict": ypred.round(2)
})
df2 = df2.reset_index(drop=True)
print("\nActual vs Predicted:")
print(df2)
```

Output:

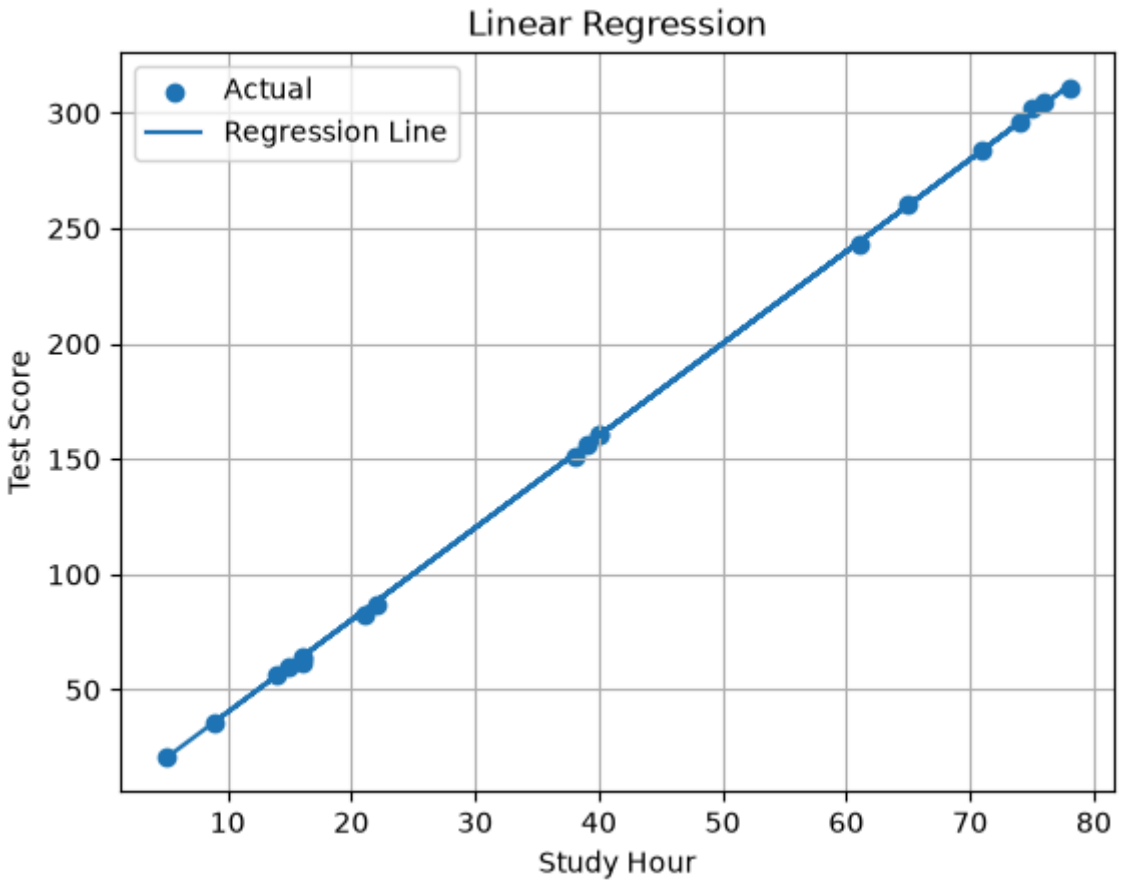
```
Actual vs Predicted:
    StudyHour  Y-Actual  Y-Predict
0          41    164.33     164.00
1          37    147.84     148.01
2           1      3.86       4.06
3          67    268.30     267.97
4          35    141.23     140.01
5          76    304.35     303.96
6          68    272.97     271.97
7          24     96.01      96.03
8          94    376.60     375.93
9          82    328.65     327.95
10         98    390.56     391.92
11          2      8.22       8.06
12         76    302.35     303.96
13          8     32.49      32.05
14         83    331.72     331.95
15         41    163.59     164.00
16         68    272.57     271.97
17         68    272.31     271.97
18         24     96.01      96.03
19         65    259.26     259.97
```

```
# Save results to CSV
df2.to_csv("linear_regression_predictions.csv", index=False)
print("\nResults saved successfully.")
```

output:

```
Results saved successfully.
```

```
plt.scatter(xtest, ytest, label="Actual")
plt.plot(xtest, ypred, label="Regression Line")
plt.xlabel("Study Hour")
plt.ylabel("Test Score")
plt.title("Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```


Linear Regression
Actual
Regression Line
Test Score
Study Hour
300
250
200
150
100
50
10
20
30
40
50
60
70
80

HOME-ASSIGNEMENT-01

Q1:

```
MARKS = np.array([ [80, 75, 90], [65, 70, 72], [90,88, 95], [55, 60,
58]])
print("Average of each student marks: ", np.mean(MARKS, axis=1))
print("Highest student Average marks: ", np.max(np.mean(MARKS,
axis=1)))
print("Average of each subject marks: ", np.mean(MARKS, axis=0))
print("Highest subject Average marks: ", np.max(np.mean(MARKS,
axis=0)))
print("student averages MORE THAN 75:",np.where(np.mean(MARKS, axis=1)
> 75)[0])
```

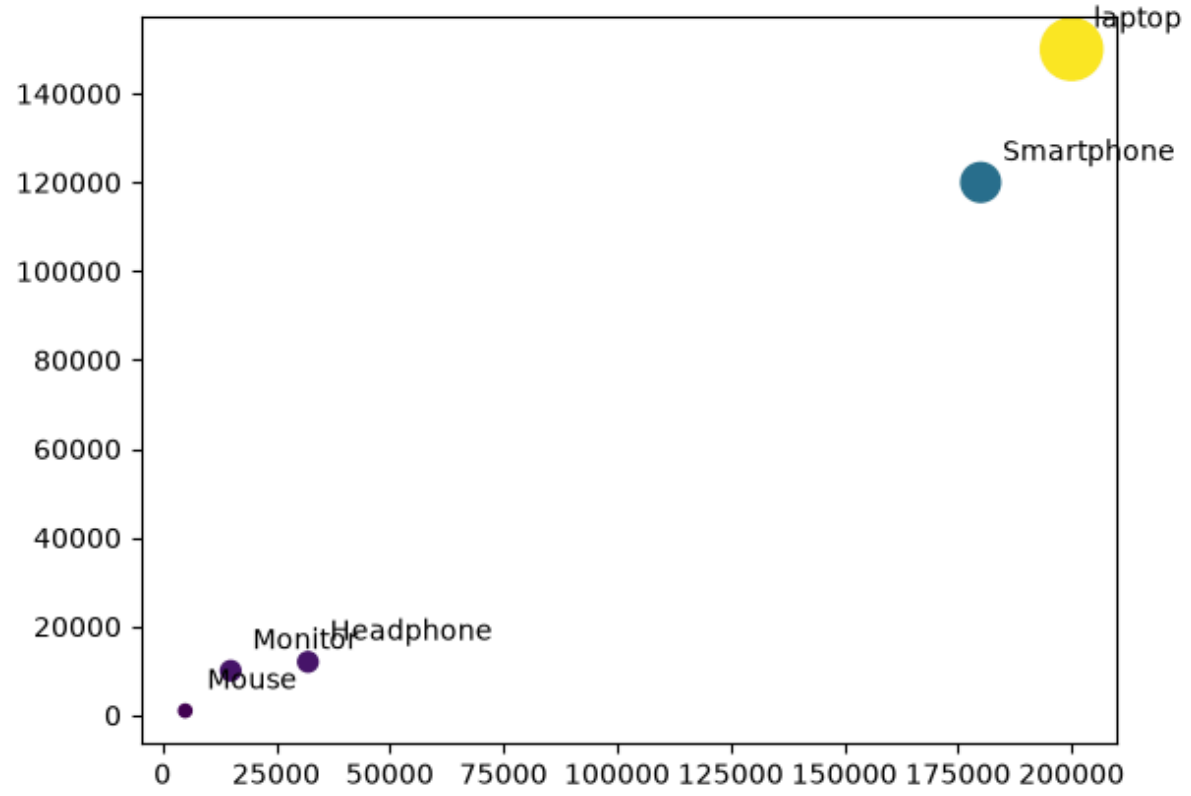
Output:

```
Average of each student marks:  [81.66666667 69.         91.         57.66666667]
Highest student Average marks:  91.0
Average of each subject marks:  [72.5  73.25 78.75]
Highest subject Average marks:  78.75
student averages MORE THAN 75: [0 2]
```

Q2:

```
data={
    'product':['laptop','Smartphone','Monitor','Headphone','Mouse'],
    'Price':[200000,180000,15000,32000,5000],
    'Sales':[150000,120000,10000,12000,1000],
    'profit':[50000,20000,5000,5000,2000]
}
df=pd.DataFrame(data)
plt.scatter(df['Price'],df['Sales'],c=df['profit'],s=df['profit'] /
100, cmap='viridis')
for i in range(len(df)):
    plt.annotate(df['product'][i], (df['Price'][i], df['Sales'][i]),
xytext=(8,8), textcoords='offset points')
```

Output:



Question 3:

```
import pandas as pd

df=pd.read_csv("Breast_Cancer_data.csv",header=None)

df
```

Output:

	0	1	2	3	4	5	6	7	8	9	...	22	23	24	25	26	27	28	29	30	31
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...	25.380	17.33	184.60	2019.0	0.16220	0.66560	0.7119	0.2654	0.4601	0.11890
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	...	24.990	23.41	158.80	1956.0	0.12380	0.18660	0.2416	0.1860	0.2750	0.08902
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...	23.570	25.53	152.50	1709.0	0.14440	0.42450	0.4504	0.2430	0.3613	0.08758
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...	14.910	26.50	98.87	567.7	0.20980	0.86630	0.6869	0.2575	0.6638	0.17300
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...	22.540	16.67	152.20	1575.0	0.13740	0.20500	0.4000	0.1625	0.2364	0.07678
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	25.450	26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...	23.690	38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...	18.980	34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	25.740	39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...	9.456	30.37	59.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039

569 rows × 32 columns

Question 4:

A company has a dataset containing 20000 customer records. Each record has:

- **Age**
- **Income**
- **Number of purchases**
- **Membership duration**
- **Website visits**
- **Customer spending {Target Var.}**

1. Total number of samples:

$$N = 20000$$

2. Number of input variables:

$$p = 5$$

3. Input vector for the i-th sample:

$$x_i = \left(Age_i, Income_i, Purchases_i, Membership_i, Visits_i\right)^T \in R^5$$

4. Output variable:

$$y_i = CustomerSpending_i$$

5. Input matrix X and its dimensions:

$$X = \left[x_1^T x_2^T \vdots x_{20000}^T\right]$$
$$X \in R^{20000\times5}$$

6. Output vector y and its dimensions:

$$y = \left[y_1 y_2 \vdots y_{20000}\right]$$
$$y \in R^{20000\times1}$$

7. Complete dataset:

$$D = \left\{\left(x_i, y_i\right)\right\} for\ i = 1, 2, \ldots, 20000$$

8. Machine learning function:

$$f: R^5 \to R$$
$$y = f(x)$$

9. Prediction equation:

$$\hat{y}_i = f\left(x_i\right)$$

10. Training dataset:

Since 16000 samples are used for training:

$$D_{train} = \{(x_i, y_i)\} \, for \, i = 1, 2, \ldots, 16000$$

$$X_{train} \in R^{16000 \times 5}$$

$$y_{train} \in R^{16000 \times 1}$$

11. Testing dataset:

Number of testing samples:

$$20000 - 16000 = 4000$$

$$D_{test} = \{(x_i, y_i)\} \, for \, i = 16001, 16002, \ldots, 20000$$

12. Dimensions of X_test:

$$X_{test} \in R^{4000 \times 5}$$

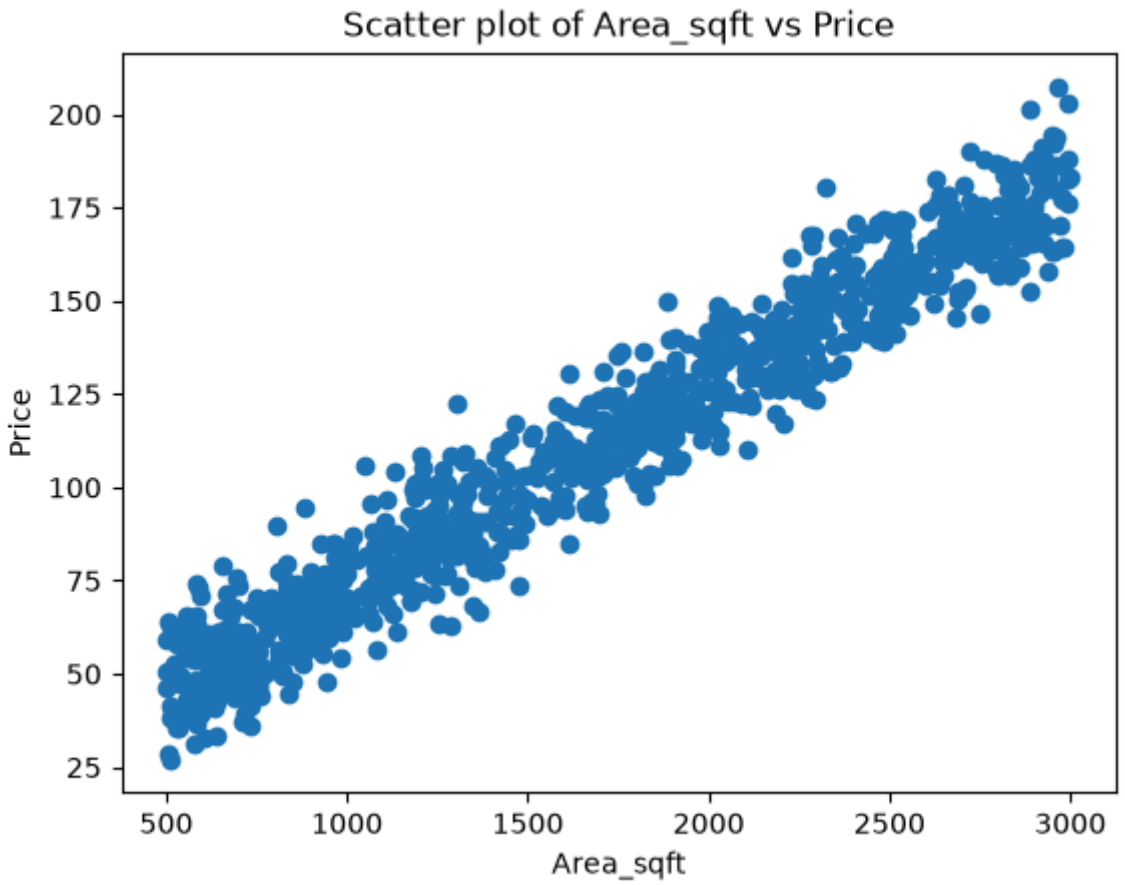
Question5:

```
import numpy as np
import matplotlib.pyplot as plt
import random
import pandas as pd
from scipy import stats

df=pd.read_csv("Simple_House_Price.csv")
dependent_variable = df['Price']
independent_variable = df['Area_sqft']

x=independent_variable
y=dependent_variable
plt.scatter(x,y)
plt.xlabel("Area_sqft")
plt.ylabel("Price")
plt.title("Scatter plot of Area_sqft vs Price")
```

Output:




```
xtrain,xtest,ytrain,ytest=train_test_split(x,y,test_size=.2,random_stat
e=42)
x_train = xtrain.values.reshape(-1, 1)
x_test = xtest.values.reshape(-1, 1)
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))

print("Training percentage:", (len(x_train) / len(df)) * 100, "%")
print("Testing percentage:", (len(x_test) / len(df)) * 100, "%")
```

Output:

```
Training samples: 800
Testing samples: 200
Training percentage: 80.0 %
Testing percentage: 20.0 %
```

#train and testing

```
lr=LinearRegression()
lr.fit(xtrain,ytrain)
ypred=lr.predict(xtest)
print("Predicted values:", ypred)
print("Dimensions of target variable:", ytest.shape)
print("Dimension of predicted variable:", ypred.shape)
```

Output:

```
 142.13456739 117.08644622  64.91650321  99.29627737 171.00266347
 141.53428562  73.1567348  165.81841181  81.88810601 168.9289628
  66.17163782  66.00792461 164.45413506  94.65773641  51.54659104
  82.48838778 144.4811234   64.04336609  67.04477495 134.44004651
  66.17163782  77.63156254 140.00629566  86.58121804  95.64001567
  62.46080506 156.81418524 140.44286422 165.05441683 180.38888752
  53.45657849 143.06227558  76.430999   167.23725963 177.66033402
 105.02623973  71.08303413 143.88084163  55.58485023 151.52079145
 101.47912017 169.09267601 114.03046629 146.00911337 115.1218877
 179.07918184 155.99561919 153.92191853 145.24511839  83.96180667
  69.71875738 157.85103558  68.02705421 105.13538187 111.52019707
 175.42292015  57.00369805 171.43923203 116.86816194  83.3615249
  52.47429923  85.9263652   77.57699147  60.98738617  46.36233938
 119.81499972  92.42032254 129.41950806 142.57113595 155.067911
  75.83071723  89.41891368  51.38287783  87.07235767 181.31659572
  55.20285274 107.04536932 111.90219456  90.78319043 119.92414186
  49.58203252 130.29264518  91.219759    50.61888285 136.73203145
  48.81803754  63.1156579  110.31963353  96.34943958 104.26224475
...
 168.98353387 117.8504412   76.70385435 114.13960843  91.27433007
  54.54799989  57.71312196 178.36975793  58.31340373 155.50447956]
Dimensions of target variable: (800,)
Dimension of predicted variable: (800,)
```

```
y_test_pred = lr.predict(xtest)
result = pd.DataFrame({
    "Area_sqft": xtest.flatten(),
    "Actual Price": ytest.values,
    "Predicted Price": y_test_pred
})
result["Residual"] = result["Actual Price"] - result["Predicted Price"]

print("\nTesting Results:")
print(result)
```

Output:

```
Testing Results:
     Area_sqft  Actual Price  Predicted Price   Residual
0         1187         86.90        83.525238   3.374762
1         1991        131.48       127.400378   4.079622
2         2404        159.27       149.938230   9.331770
3         1819        100.71       118.014154 -17.304154
4         1709        118.42       112.011337   6.408663
..         ...           ...              ...        ...
195       1888        139.75       121.779558  17.970442
196       1104         91.04        78.995839  12.044161
197       1353         90.76        92.584036  -1.824036
198       2789        163.61       170.948092  -7.338092
199       1203         72.26        84.398375 -12.138375

[200 rows x 4 columns]
```

```
mae = mean_absolute_error(ytest, y_test_pred)
rmse = np.sqrt(mean_squared_error(ytest, y_test_pred))
r2 = r2_score(ytest, y_test_pred)

print("\nModel Evaluation:")
print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
if rmse > mae:
    print("\nRMSE is larger than MAE.")
    print("RMSE:", rmse)
    print("MAE:", mae)
else:
    print("\nMAE is larger than RMSE.")
    print("MAE:", mae)
    print("RMSE:", rmse)
```

Output:

```
Model Evaluation:
MAE: 7.7660138381003385
RMSE: 10.155207775154258
R2 Score: 0.9410312875961606

RMSE is larger than MAE.
RMSE: 10.155207775154258
MAE: 7.7660138381003385
```

—————X—————